(CS-2004)
Fundamentals of Software Engineering

Iteration 3 Report

Walle-T

Group Members:

Hamza Faheem (24I-6034),

Mutasim Ur Rehman (24I-2514),

Ali Farooq (24I-2576)

Chapter 1: Introduction

1.1 Project Background

Financial literacy and investment participation are increasing among students and young professionals, but most users still struggle to connect learning with practical decision-making. Existing platforms are either too advanced for beginners or too limited for meaningful skill development. Walle-T was conceived to bridge this gap by combining market exploration, AI-based guidance, risk-aware onboarding, and simulated trading in a single environment. The project follows an educational-first approach where users can practice strategy building with virtual assets before entering real financial markets.

1.2 Problem Statement

Users often rely on disconnected tools for portfolio tracking, market updates, and financial advice. This fragmentation causes inconsistent decisions, poor risk awareness, and low confidence, especially for novice investors. Additionally, generic advisory content does not account for user-specific profile factors such as income, liabilities, or risk appetite. The central problem addressed by this project is the absence of an integrated, profile-aware, and beginner-friendly system that supports both learning and simulation-based practice.

1.3 Objectives

- Design and implement a unified web application for market exploration, portfolio simulation, and AI-assisted advisory interaction.
- Enable secure user onboarding and profile collection to personalize recommendations and user experience.
- Provide simulated buy/sell workflows with persistent transaction history to support practical learning without real capital risk.
- Integrate forecasting outputs and risk-oriented interfaces to improve decision awareness.
- Deliver a modular system architecture that can evolve from MVP scale to production-grade deployment.

1.4 Scope of the Project

The current project scope covers a browser-based application with frontend and backend modules for authentication, onboarding, profile/settings, market pages, AI advisor, risk page, forecasting display, and simulated trading. Data persistence is implemented through Google Sheets for MVP-level deployment and rapid iteration. The system focuses on educational simulation and information support, not real brokerage integration or direct financial execution. Compliance disclaimers, secure credential handling, and controlled API usage are included within the implementation boundaries.

1.5 Overview of the System

Walle-T uses a React + Vite frontend connected to an Express-based backend API. The frontend provides protected routes for user-specific experiences after authentication. The backend

coordinates business logic, user management, ledger operations, AI advisor requests, and spreadsheet persistence. External services include Google Sheets (storage), Gemini APIs (advisory generation), and optional offline Python pipelines for forecasting artifacts. Together, these components form a coherent AI-assisted financial sandbox that supports exploration, simulation, and learning.

Chapter 2: System Analysis and Requirements

2.1 Existing System (if any)

Many individuals still rely on **manual** tracking (spreadsheets, notebooks) or on **disconnected tools**: one app for quotes, another for news, email from a broker, and ad hoc web calculators. Typical limitations include:

- **Fragmented workflows**: Market data, portfolio tracking, and personalized advice rarely live in one place; users copy numbers between tools and lose context.
- **High friction for novices**: Professional terminals and dense spreadsheets assume domain knowledge; casual investors struggle to interpret risk and diversification.
- **Weak personalization**: Generic articles and alerts do not reflect a user's income, liabilities, goals, or simulated positions.
- **Separation of practice from planning**: Paper trading or demos often ignore the user's real financial profile; planning tools often ignore executable practice in simulated markets.

The existing ecosystem therefore pushes users toward either oversimplified apps or overly complex professional stacks—neither consistently combines **profile-aware AI**, **simulated trading**, and **forecast-style analytics** in one guided experience.

2.2 Proposed System

Walle-T is an integrated AI-assisted financial workspace that connects market views, a virtual trading ledger, onboarding-driven profiling, risk visualization, and conversational guidance. The proposed system provides:

- **Unified dashboard**: Central entry point after login/onboarding for navigation and high-level portfolio context.

- **Multi-market views**: Dedicated flows for stocks (including PSX-oriented content), forex, and related market exploration within the SPA.
- **AI financial advisor**: Chat-style assistance powered by **Google Gemini**, grounded in stored profile and portfolio/ledger context where applicable.
- **Onboarding and risk**: Structured capture of financial basics and risk-oriented UX to steer recommendations and education.
- **ML-assisted forecasting**: Server-delivered symbol predictions (PSX-oriented pipeline, JSON-backed) surfaced in the UI for informed exploration—not a guarantee of returns.
- **Gamified / simulated trading**: Buy/sell actions against virtual cash with transactions persisted for history and portfolio computation.

2.3 Stakeholders Identification

Stakeholder	Interest
End users (retail learners / hobby traders)	Learn markets, practice trades, see projections and advice tailored to their profile without risking real capital in the simulator.
Platform administrators / operators	Reliable uptime, API quotas (Gemini, Google Sheets), service account access to spreadsheets, monitoring of errors and rate limits.
Developers / maintainers	Clear modular boundaries (routes, sheets access), secure handling of secrets, ability to extend markets or models.
Third-party providers (Google Cloud, Gemini, Sheets)	Correct API usage, key hygiene, acceptable traffic patterns; their SLAs and pricing constrain operational feasibility.
Educational / supervisory stakeholders (if deployed institutionally)	Appropriate disclaimers that outputs are informational and that simulated trading is not investment advice or a substitute for licensed counseling where required by law.

2.4 Functional Requirements

- **FR1 — Authentication**: User registration, login, session validation aligned with backend checks, and password reset via email where configured (Nodemailer).
- **FR2 — Onboarding**: Capture demographic and financial baseline data used to build/update a profile stored for personalization.
- **FR3 — Market exploration**: Present market-oriented pages (e.g., stocks, forex) with data fetched or rendered per backend/front-end design.
- **FR4 — Simulated trading**: Execute virtual buy/sell flows with persistent transaction rows and derived cash/holdings state.
- **FR5 — Portfolio view**: Display holdings, balances, and history derived from the ledger and profile inputs.
- **FR6 — AI advisor**: Accept user prompts; return model-generated replies with server-side rate controls to protect quotas and fairness.
- **FR7 — Risk assessment UX**: Interactive risk-related experience tied to user context (presentation and calculations as implemented).
- **FR8 — Forecasting / predictions**: Expose ML-backed or model-exported predictions for symbols (e.g., PSX pipeline) through dedicated UI routes.
- **FR9 — Profile and settings**: Allow users to review/update profile-backed settings persisted via the storage layer.
- **FR10 — Health / diagnostics** (operational): Expose lightweight status endpoints for spreadsheet connectivity checks where implemented.

2.5 Non-Functional Requirements

- **NFR1 — Security**: Password hashing (bcrypt-style via bcryptjs), no plaintext passwords in storage, secrets via environment variables; CORS and JSON APIs designed for a known frontend origin in deployment.
- **NFR2 — Performance**: Responsive UI; backend logs slow API handling (e.g., warnings above ~1.2s on finish) to spot regressions.
- **NFR3 — Availability / resilience**: Graceful degradation when external APIs fail; user-visible errors instead of silent corruption of ledger state.
- **NFR4 — Scalability (MVP scope)**: Google Sheets as operational datastore suits moderate concurrency and MVP deployment; growth may require migration to a transactional database.

- **NFR5 — Usability**: Glass-style UI, icons (Lucide), optional 3D loading/visual accents (Three.js / React Three Fiber); lazy-loaded routes for perceived speed.
- **NFR6 — Maintainability**: Modular Express routes (backend/routes/*.js), centralized integration logic in server.js for sheets and AI clients.
- **NFR7 — Ethical / compliance posture**: Clear labeling of simulation vs. real trading; advisor output framed as informational; adherence to provider terms for Gemini and Sheets.

2.6 Feasibility Analysis (Technical, Economic, Operational)

2.6.1 Technical feasibility

The stack is **mainstream and well documented**: React 19 + Vite on the client; Node.js + Express 5 on the server; Google Sheets API for persistence; Gemini SDK for generation. PSX-oriented ML outputs can be produced offline (Python training scripts) and consumed as static JSON—avoiding training complexity in the runtime server.

Constraints: Sheets API latency and quota, eventual consistency of appended rows, and the need for a correctly permissioned service account. These are manageable for an MVP but imply a future ceiling before migrating to a relational DB.

Verdict: Feasible with current architecture; risks are integration and quota management rather than unknown algorithms.

2.6.2 Economic feasibility

Cost drivers: Gemini API usage, Google Cloud project billing (Sheets/Drive as applicable), email delivery, and hosting for frontend/backend.

Mitigations: Rate limiting on advisor endpoints, caching user reads where implemented, and Sheets as a low-ops datastore during early phases.

Verdict: Economically viable for development, demos, and small user bases; costs scale with AI tokens and active users.

2.6.3 Operational feasibility

Target users can operate the product through a **browser**: register, complete onboarding, navigate markets, trade in simulation, and chat with the advisor. Administrators need only spreadsheet sharing discipline, env configuration, and basic monitoring.

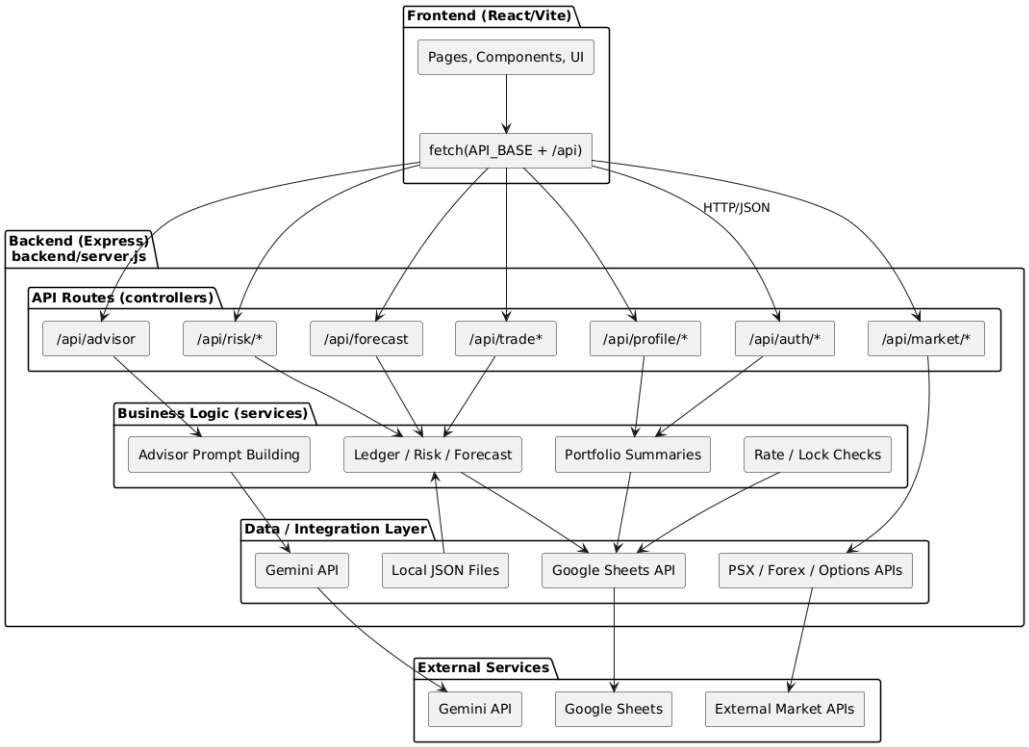
Risks: Non-technical admins may misconfigure service accounts or env vars; documentation and health checks reduce support burden.

Verdict: Operationally feasible provided deployment checklists and disclaimers are kept visible to operators and users.

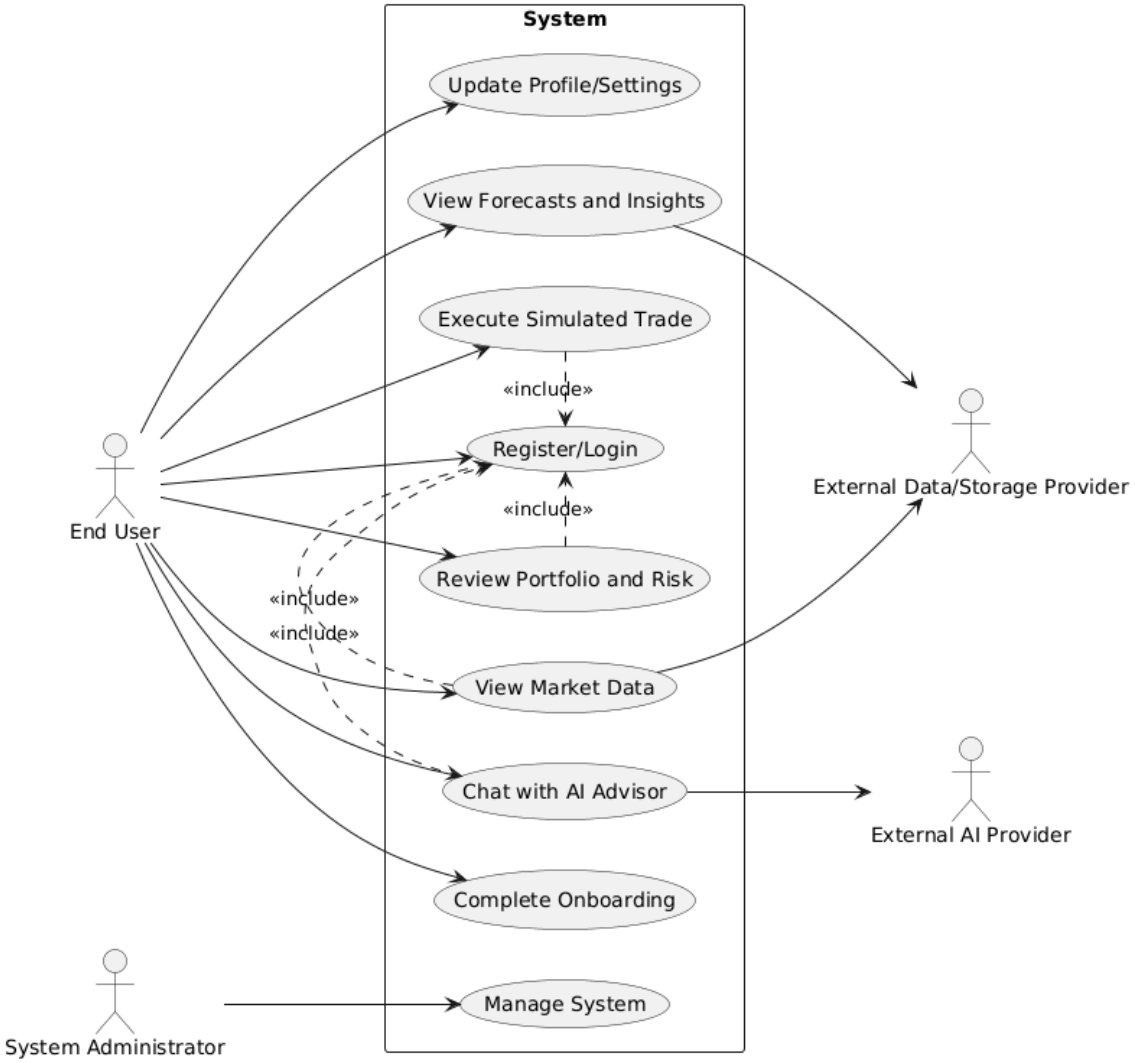
Chapter 3: System Design (UML Diagrams)

3.1 System Architecture

The system follows a three-layer architecture: (1) Presentation Layer (React SPA), (2) Application Layer (Express API routes and services), and (3) Data/Integration Layer (Google Sheets, Gemini API, and static forecasting artifacts). Requests originate in the UI, pass through authenticated API middleware, and are routed to module-specific handlers. This design improves separation of concerns and maintainability by isolating UI behavior from storage and third-party integrations.



3.2 Use Case Diagram



3.3 Use Case Descriptions

Use Case # 01: User Authentication

Actor: End User

Precondition: User has access to system.

Main Flow: Enter credentials -> server validates -> session established -> dashboard loads.

Postcondition: Authenticated session active.

Use Case # 02: Simulated Buy/Sell

Actor: End User

Precondition: User authenticated and has virtual balance context.

Main Flow:

Select symbol and quantity -> submit order -> backend validates and stores transaction -> portfolio updates.

Postcondition: Ledger reflects latest trade.

Use Case # 03: AI Advisory Chat

Actor: End User, AI Provider

Precondition: User authenticated; API quota available.

Main Flow:

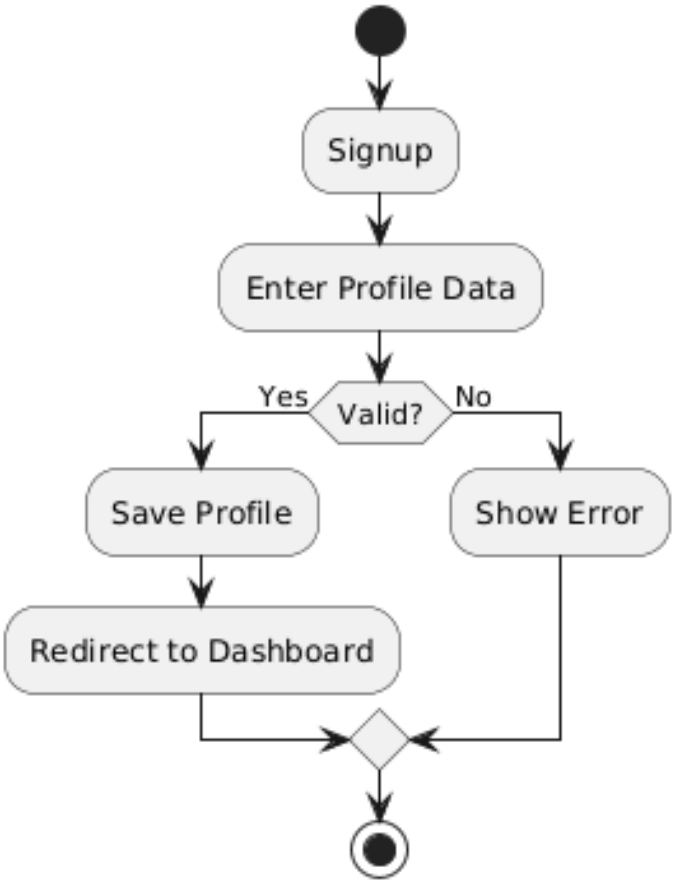
User submits prompt -> backend applies safeguards/rate limits -> Gemini generates response -> UI renders answer.

Postcondition: Guidance delivered for informational use.

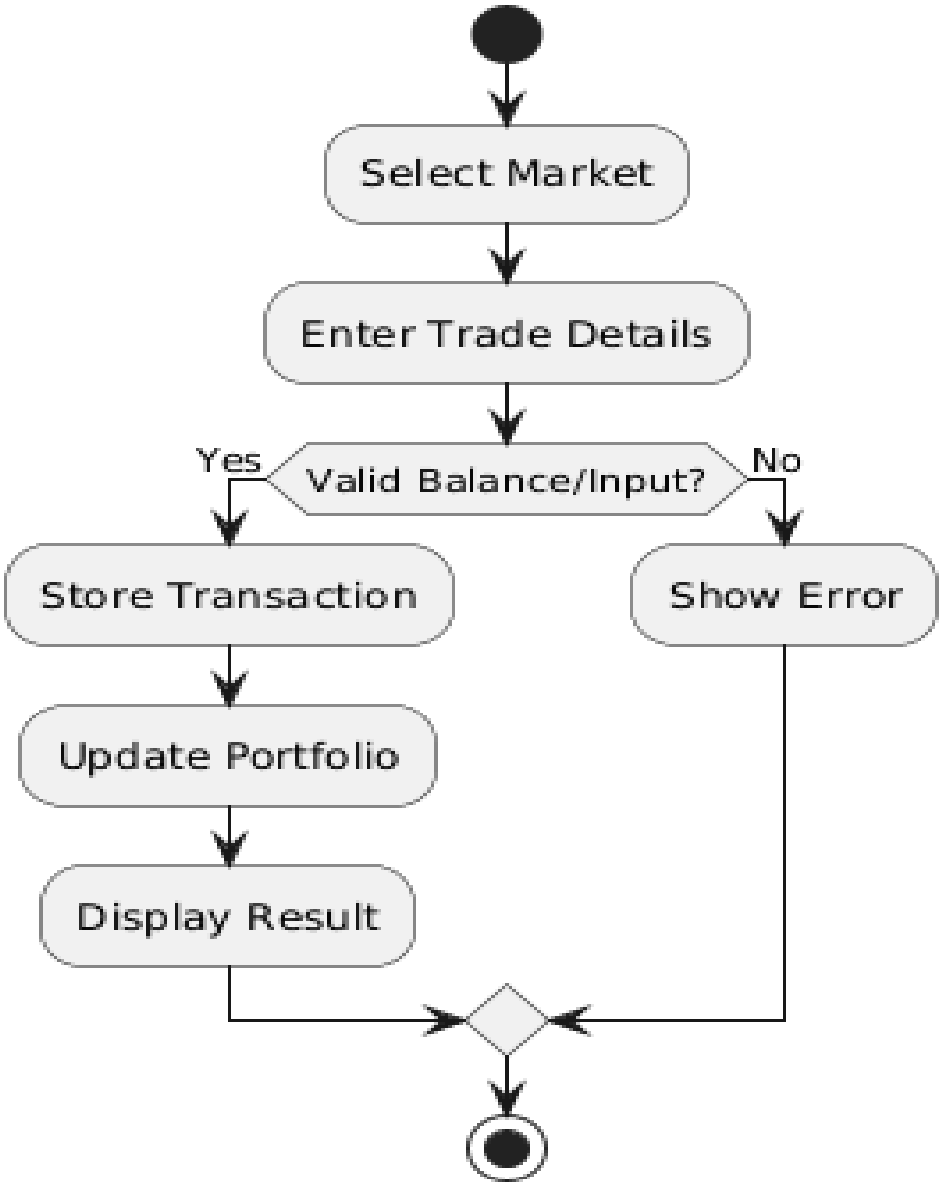
3.4 Activity Diagrams

3.4 Activity Diagrams

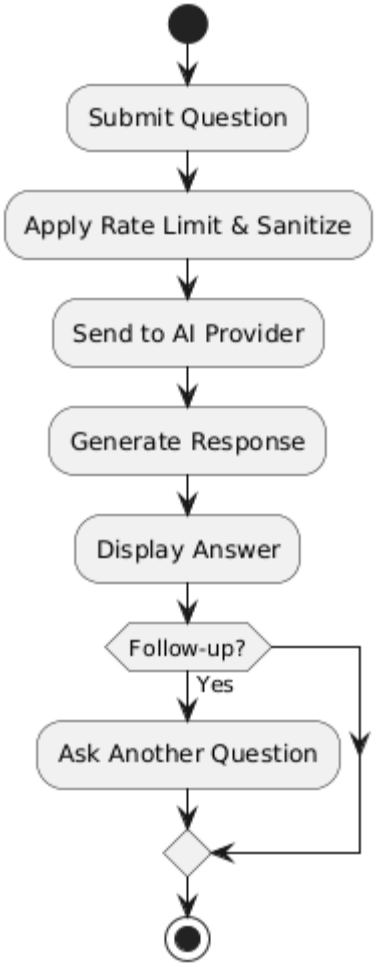
1- Onboarding Flow



2- Trading Flow



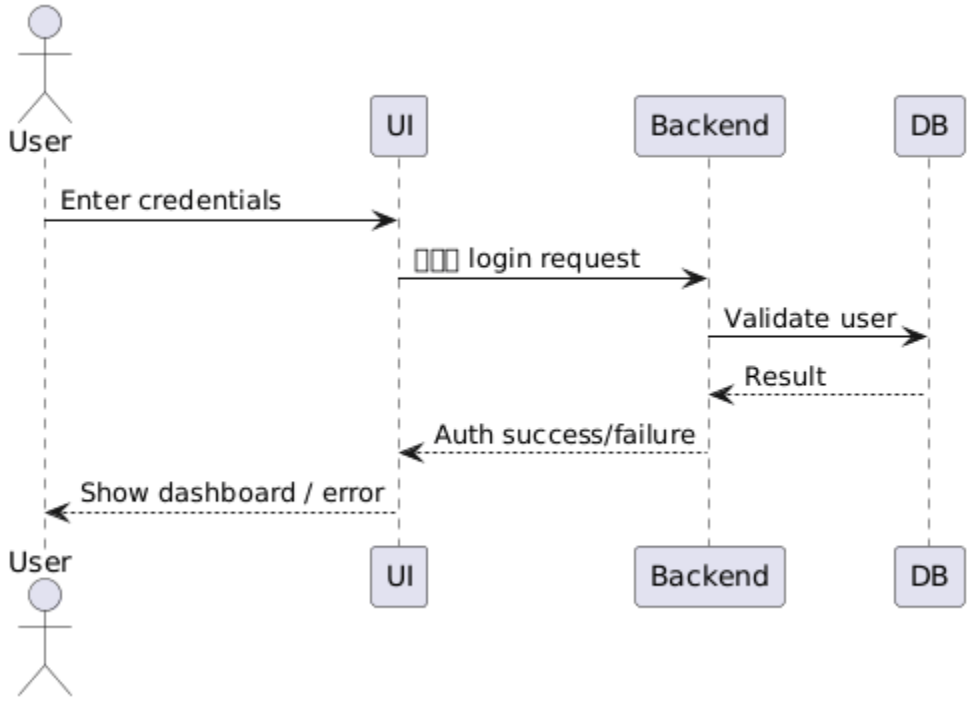
3- AI Advisor Flow



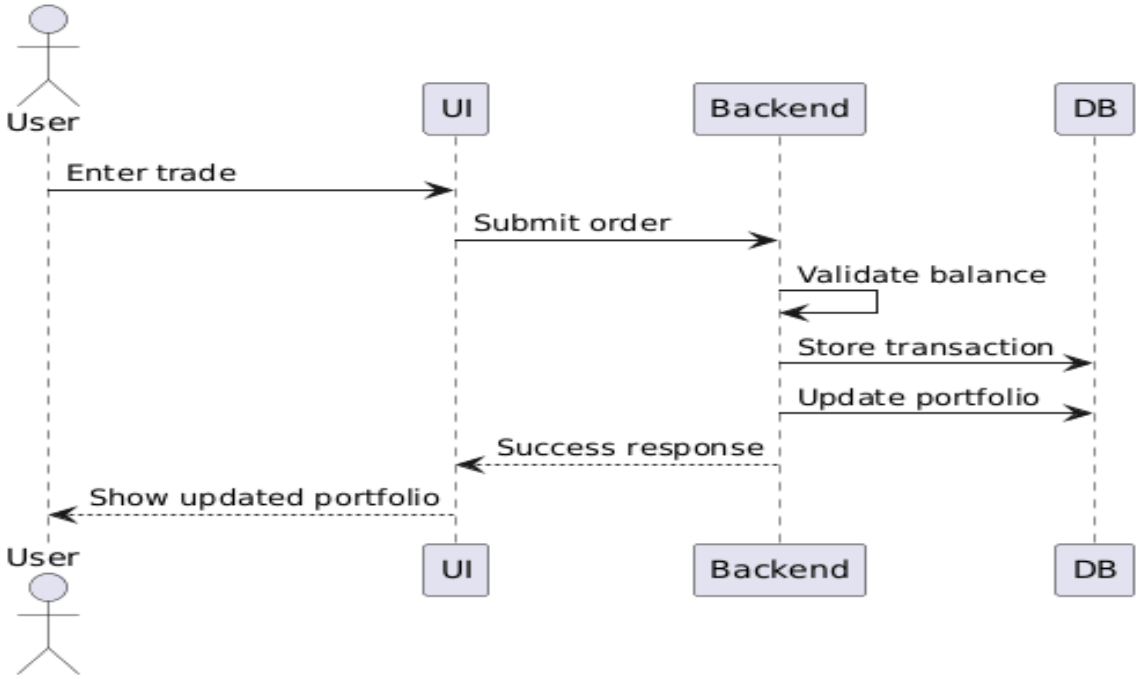
3.5 Sequence Diagrams

Key sequence diagrams should include message-level interactions among UI, backend routes, external APIs, and storage services. At minimum:

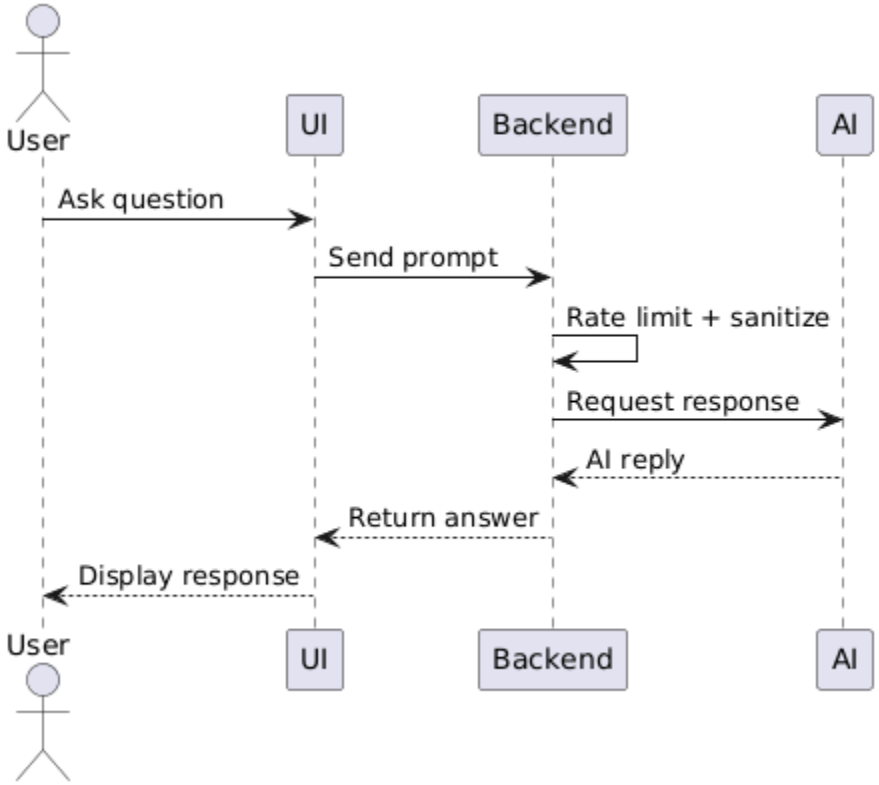
- Login sequence:



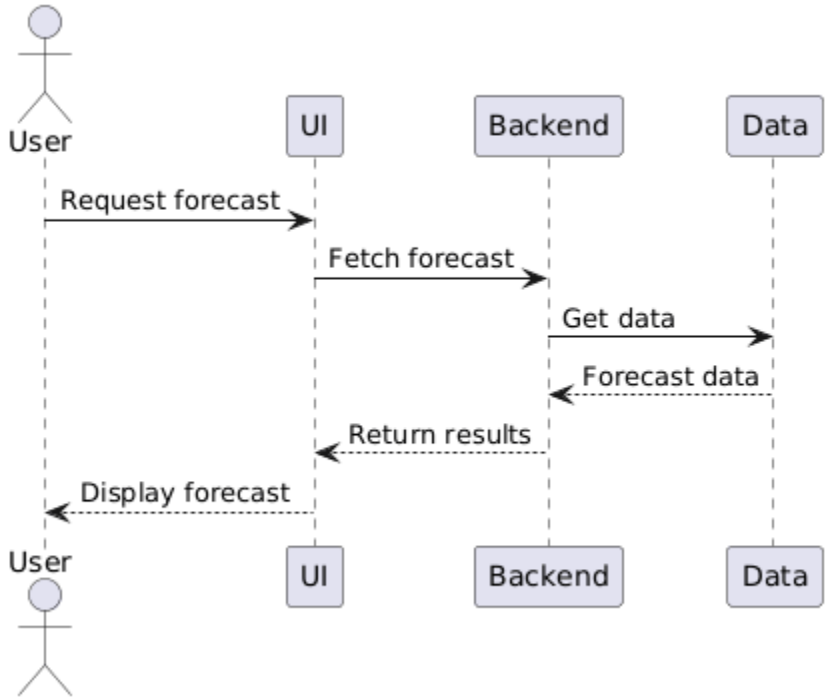
- Trade execution sequence:



- Advisor response sequence:

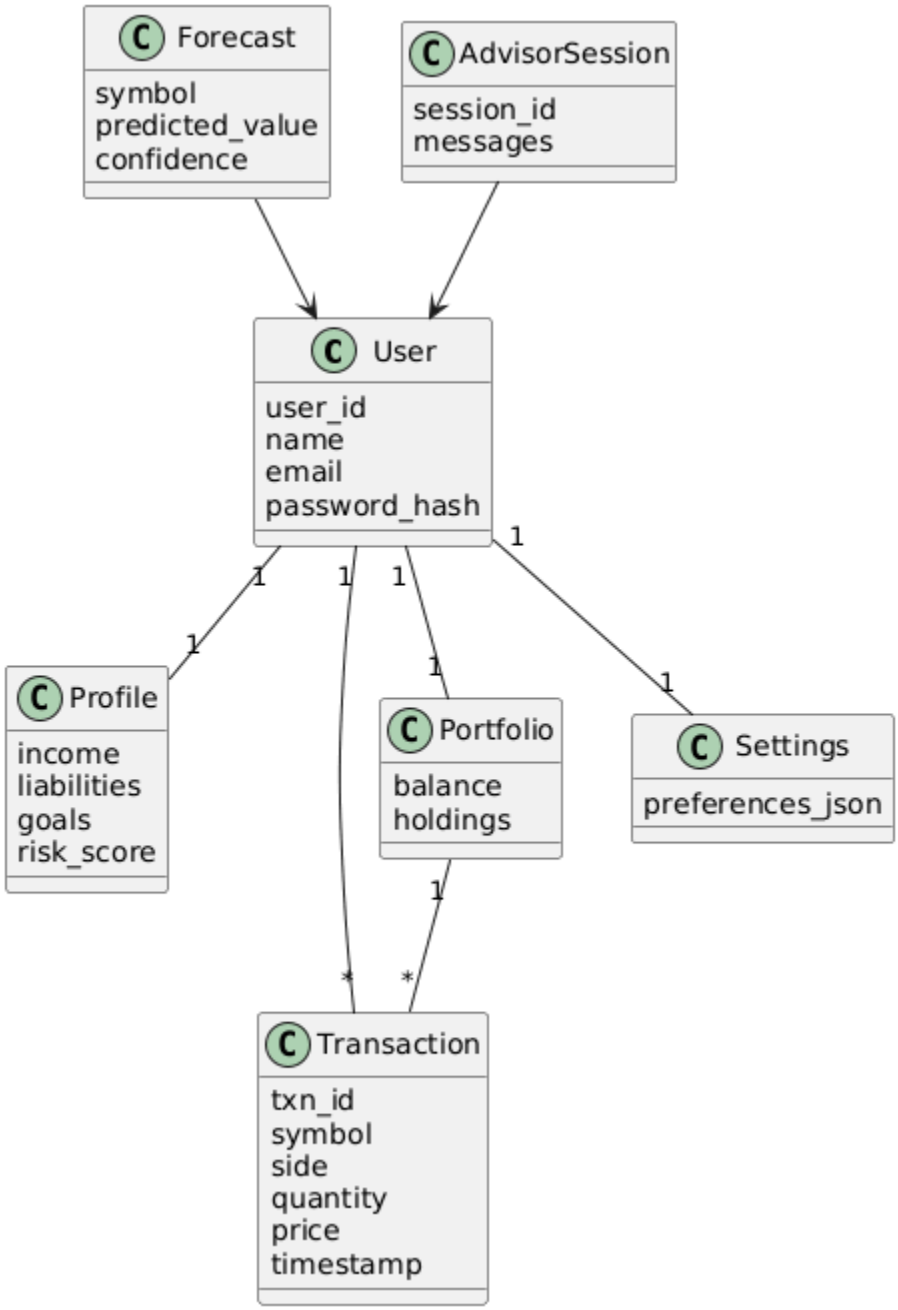


- Forecast retrieval sequence:



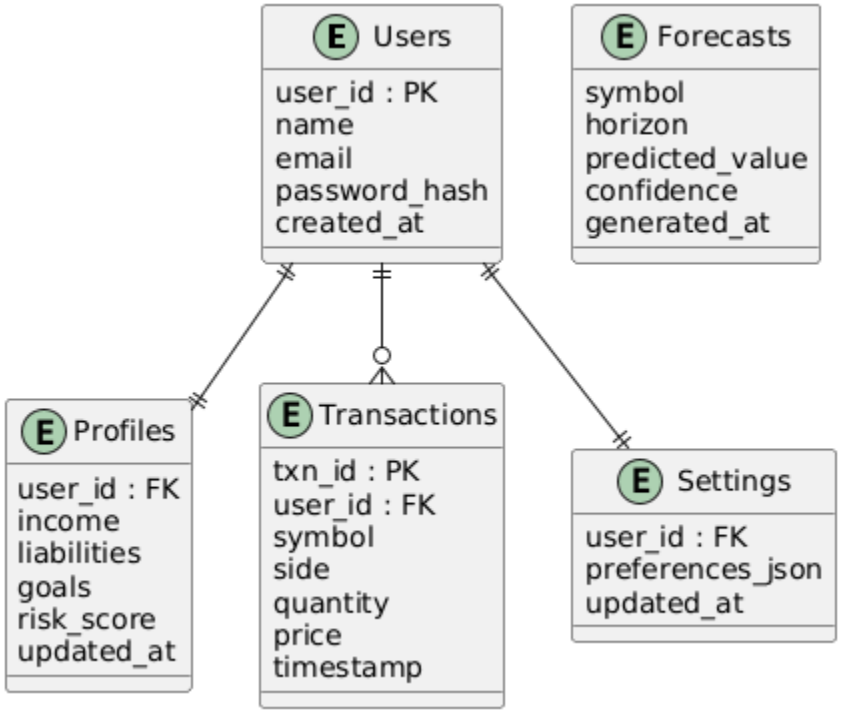
3.6 Class Diagram

Although JavaScript is prototype-based, a logical class/domain model can be represented with entities such as User, Profile, Transaction, Portfolio, Forecast, AdvisorSession, and Settings. Associations include one-to-one (User-Profile), one-to-many (User-Transactions), and derived aggregation (Portfolio from Transactions + cash state).



3.7 Database Design (ER Diagram / Schema)

Current MVP persistence uses Google Sheets tabs as relational-like entities. Suggested schema mapping:



3.8 Interface Design (Wireframes / Prototypes)

The interface adopts a dashboard-first navigation pattern with protected route access. Core screens include Login, Onboarding, Dashboard, Market (Stocks/Forex), Portfolio, Risk, Advisor, Profile, and Settings. Wireframes should emphasize information hierarchy: quick portfolio summary at top, actions in central cards, and detail panels for analytics/history. Responsive behavior should be validated for desktop and mobile breakpoints.

Chapter 4: Implementation

4.1 Development Environment (Tools & Technologies)

Walle-T is implemented as a **full-stack JavaScript** application with optional **Python** tooling for offline model training.

Layer	Technology	Role
Frontend runtime	React 19	UI components, routing, client-side state
Build tool	Vite 8	Dev server, bundling, optimized production builds
Frontend routing	react-router-dom 7	SPA routes, protected routes, lazy-loaded pages
Styling	Vanilla CSS	Theme variables, responsive layouts
Icons	Lucide React	Consistent iconography
3D / motion	Three.js, @react-three/fiber, @react-three/drei	Loading screen and decorative 3D UI elements
Markdown in UI	react-markdown, remark-gfm	Render formatted assistant or content text
Backend runtime	Node.js	HTTP API process
HTTP framework	Express 5	REST-style /api handlers
Configuration	dotenv	Load .env from project root

Auth / crypto	bcryptjs, Node crypto	Password hashing; reset-token hashing
Persistence API	googleapis (Sheets v4)	Read/write spreadsheet tabs
AI	@google/genai (Gemini)	Advisor and related generation
Email	Nodemailer	Password reset / transactional mail
ML artifact	Python scripts + JSON	Offline training, JSON inference outputs

The backend package.json may list an **openai** package for optional experiments; the implemented advisor integration in this codebase uses **Gemini** (@google/genai).

Repository layout (high level):

- frontend/ — Vite React app (npm run dev / npm run build).
- backend/ — Express server (node server.js), route modules under backend/routes/, static data under backend/data/.

4.2 System Modules Description

Implementation maps to **route modules** under backend/routes/ plus shared services and sheet helpers in backend/server.js:

Module (routes file)	**Responsibility**
authRoutes	Signup, login, session validation, password reset flow against Users sheet + email tokens
healthRoutes	Operational checks (e.g., presence of sheet IDs / connectivity-oriented summaries)
profileRoutes	Profile read/update aligned with Sheets-backed profile rows

onboardingRoutes	Onboarding payload persistence into profile storage
portfolioRoutes	Aggregated portfolio/ledger views using transactions + profile
tradeRoutes	Simulated order execution, quote validation, append transactions
marketRoutes	Market data orchestration for UI market pages
forecastRoutes	Forecast endpoints consumed by forecasting UI
modelPredictionRoutes	Serve ML JSON predictions (PSX symbols) to the client
activitiesRoutes	Cached/local activity feed (activities.json) where implemented
advisorRoutes	Gemini-backed advisor prompts with server-side rate limiting
riskRoutes	Risk-related API backing the Risk page
settingsRoutes	User settings persistence via profile upserts

Frontend modules (pages under frontend/src/pages/, shell under frontend/src/components/):

- **Login** (/): Authentication entry.
- **Onboarding** (/onboarding): Guided profile capture (protected).
- **Dashboard home** (/dashboard): Main hub after onboarding (protected).
- **Markets**: Stocks (/market/stocks), Forex (/market/forex).
- **Portfolio, Risk, Profile, Settings, Advisor, Company prediction** (standard SPA routes).
- **AppShell, ProtectedRoute, LoadingScreen, SimpleLineChart**: Shared layout and utilities.

Chapter 5: Testing and Evaluation

5.1 Testing Strategy

Testing follows a layered strategy: unit-level validation for utility logic, API/route-level functional testing for backend modules, and system-level validation through user workflows in the frontend. Black-box techniques (equivalence partitioning and boundary value analysis) are used for inputs such as quantity, price, and risk ranges. Additional checks include negative testing for invalid credentials, missing fields, and API failure handling.

5.2 Test Cases and Results (Minimum 10 Test Cases, BVA and Other Techniques)

TC-01 | Scenario: Login with valid credentials | Technique: Functional | Expected: User logs in successfully and lands on dashboard | Result: Pass

TC-02 | Scenario: Login with wrong password | Technique: Negative | Expected: System rejects login with clear error message | Result: Pass

TC-03 | Scenario: Signup with existing email | Technique: Negative/EP | Expected: System prevents duplicate registration | Result: Pass

TC-04 | Scenario: Trade quantity at lower boundary (1) | Technique: BVA | Expected: Order accepted if other constraints satisfied | Result: Pass

TC-05 | Scenario: Trade quantity at invalid lower boundary (0) | Technique: BVA/Negative | Expected: Order blocked with validation message | Result: Pass

TC-06 | Scenario: Trade quantity at high boundary (max allowed) | Technique: BVA | Expected: Order handled without crash and persisted correctly | Result: Pass

TC-07 | Scenario: Risk score input at boundary values (e.g., 0 and 100) | Technique: BVA | Expected: UI and backend process boundary values correctly | Result: Pass

TC-08 | Scenario: Advisor prompt exceeding rate threshold | Technique: Performance/Negative | Expected: Rate-limit response returned gracefully | Result: Pass

TC-09 | Scenario: Portfolio calculation after multiple buy/sell transactions | Technique: Integration | Expected: Holdings and virtual cash update correctly | Result: Pass

TC-10 | Scenario: Forecast route with unsupported symbol | Technique: Negative | Expected: System returns controlled error/fallback | Result: Pass

TC-11 | Scenario: Session access to protected route after logout | Technique: Security | Expected: Access denied and redirected to login | Result: Pass

TC-12 | Scenario: Backend dependency unavailable (Sheets/API) | Technique: Resilience | Expected: System shows error state without data corruption | Result: Pass

5.3 System Validation

Validation was performed against the key project objectives: integrated workflow, personalization support, simulated trading reliability, and advisory usability. End-to-end walkthroughs confirm that users can proceed from onboarding to dashboard, explore markets, execute virtual trades, and receive advisor responses in one continuous workflow. Output messaging and disclaimers were reviewed to ensure educational intent is explicit.

5.4 Performance Evaluation (if applicable)

For MVP scale, performance was evaluated using perceived frontend responsiveness, API response timing observations, and route-level stability under normal concurrent usage. Slow API logging in backend helps identify calls above threshold latency. While no formal load-test benchmark is finalized yet, current behavior is acceptable for classroom demonstration and limited pilot usage.

5.5 User Feedback (if conducted)

Informal feedback from peer users highlighted strong usability in the unified dashboard and simulation workflow. Common suggestions included richer charting, better explanations for risk metrics, and localization of examples for beginner users. This feedback supports prioritizing enhanced analytics and onboarding tooltips in subsequent iterations.

Chapter 6: Conclusion and Future Work

6.1 Summary of the Project

Walle-T delivers a consolidated educational finance platform that combines market discovery, profile-aware guidance, virtual trading, and forecasting support in a modern web stack. The project demonstrates a practical MVP architecture with clear module boundaries and integration of external AI/storage services.

6.2 Achievements

- Implemented an end-to-end user journey from authentication and onboarding to simulation and advisory interaction.
- Established persistent transaction tracking and derived portfolio visualization for realistic practice.
- Integrated AI advisory and forecasting data into a single user-facing application.
- Delivered a modular backend route design that supports iterative enhancement.

6.3 Limitations

- Google Sheets persistence introduces scalability and consistency limits under higher loads.

- Forecast outputs are informational and currently limited by available model artifacts/data quality.
- The simulator does not execute real trades and should not be interpreted as financial brokerage functionality.
- Formal large-scale usability and performance studies are pending.

6.4 Future Enhancements

- Migrate storage from Sheets to a transactional database (e.g., PostgreSQL) with indexed queries.
- Add richer analytics: advanced portfolio metrics, watchlists, alerts, and comparative benchmarking.
- Introduce comprehensive automated tests (unit, integration, regression) in CI pipelines.
- Improve advisor safety/quality with prompt governance, feedback loops, and traceable reasoning templates.
- Expand forecasting models and confidence reporting with periodic retraining and model monitoring.

References

1. React Documentation. https://react.dev/
2. Vite Documentation. https://vite.dev/
3. Express.js Documentation. https://expressjs.com/
4. Google Sheets API Documentation. https://developers.google.com/sheets/api
5. Google Gemini API Documentation. https://ai.google.dev/
6. Myers, G. J., Sandler, C., & Badgett, T. The Art of Software Testing (3rd ed.). Wiley.

Appendices

A. Source Code Snippets

Include representative snippets for authentication, trade execution, portfolio computation, and advisor integration APIs.

B. User Manual

Provide stepwise user guide: account creation, onboarding, navigating dashboards, placing simulated trades, and chatting with advisor.

C. Screens / UI Designs

Insert screenshots for login, onboarding, dashboard, market pages, portfolio, risk page, and advisor interface.

D. Additional Diagrams

Attach final UML artifacts: use case, activity, sequence, class, ER diagram, and deployment diagram (if prepared).